

OSGi Working Group
OSGi Specification repo for
`org.osgi.service.rest`

Release 1.0.1-SNAPSHOT
September 2026



Copyright © 2000, 2026 Eclipse Foundation

LICENSE

Eclipse Foundation Specification License – v1.0

By using and/or copying this document, or the Eclipse Foundation document from which this statement is linked, you (the licensee) agree that you have read, understood, and will comply with the following terms and conditions:

Permission to copy, and distribute the contents of this document, or the Eclipse Foundation document from which this statement is linked, in any medium for any purpose and without fee or royalty is hereby granted, provided that you include the following on ALL copies of the document, or portions thereof, that you use:

- link or URL to the original Eclipse Foundation document.
- All existing copyright notices, or if one does not exist, a notice (hypertext is preferred, but a textual representation is permitted) of the form: "Copyright © [date-of-document] Eclipse Foundation, Inc. <<url to this license>>"

Inclusion of the full text of this NOTICE must be provided. We request that authorship attribution be provided in any software, documents, or other items or products that you create pursuant to the implementation of the contents of this document, or any portion thereof.

No right to create modifications or derivatives of Eclipse Foundation documents is granted pursuant to this license, except anyone may prepare and distribute derivative works and portions of this document in software that implements the specification, in supporting materials accompanying such software, and in documentation of such software, PROVIDED that all such works include the notice below. HOWEVER, the publication of derivative works of this document for use as a technical specification is expressly prohibited.

The notice is:

"Copyright © [date-of-document] Eclipse Foundation. This software or document includes material copied from or derived from [title and URI of the Eclipse Foundation specification document]."

Disclaimers

THIS DOCUMENT IS PROVIDED "AS IS," AND THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION MAKE NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NON-INFRINGEMENT, OR TITLE; THAT THE CONTENTS OF THE DOCUMENT ARE SUITABLE FOR ANY PURPOSE; NOR THAT THE IMPLEMENTATION OF SUCH CONTENTS WILL NOT INFRINGE ANY THIRD PARTY PATENTS, COPYRIGHTS, TRADEMARKS OR OTHER RIGHTS.

THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION WILL NOT BE LIABLE FOR ANY DIRECT, INDIRECT, SPECIAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF ANY USE OF THE DOCUMENT OR THE PERFORMANCE OR IMPLEMENTATION OF THE CONTENTS THEREOF.

The name and trademarks of the copyright holders or the Eclipse Foundation may NOT be used in advertising or publicity pertaining to this document or its contents without specific, written prior permission. Title to copyright in this document will at all times remain with copyright holders.

Preface

Implementation Requirements

An implementation of a Specification: (i) must fully implement the Specification including all its required interfaces and functionality; (ii) must not modify, subset, superset or otherwise extend the OSGi Name Space, or include any public or protected packages, classes, Java interfaces, fields or methods within the OSGi Name Space other than those required and authorized by the Specification. An implementation that does not satisfy limitations (i) (ii) is not considered an implementation of the Specification and must not be described as an implementation of the Specification. "OSGi Name Space" shall mean the public class or interface declarations whose names begin with "org.osgi" or any recognized successors or replacements thereof. An implementation of a Specification must not claim to be a compatible implementation of the Specification unless it passes the Technology Compatibility Kit ("TCK") for the Specification.

Feedback

This specification can be downloaded from the OSGi Documentation web site:

<https://docs.osgi.org/specification/>

Comments about this specification can be raised at:

<https://github.com/osgi/osgi/issues>

Table of Contents

1	Introduction	5
1.1	Reader Level	5
1.2	Version Information	5
1.3	References	5
137	REST Management Service Specification	7
137.1	Introduction	7
137.2	Interacting with the REST Management Service	8
137.3	Resources	10
137.4	Representations	15
137.5	Clients	20
137.6	Extending the REST Management Service	21
137.7	XML Schema	22
137.8	Capabilities	26
137.9	Security	27
137.10	org.osgi.service.rest	27
137.11	org.osgi.service.rest.client	28
137.12	JavaScript Client API	33
137.13	References	36

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT

1 Introduction

This document contains a specification developed by the OSGi working group.

1.1 Reader Level

This specification is written for the following audiences:

- Application developers
- Framework and system service developers (system developers)
- Architects

This specification assumes that the reader has at least one year of practical experience in writing Java programs. Experience with embedded systems and server-environments is a plus. Application developers must be aware that the OSGi environment is significantly more dynamic than traditional desktop or server environments.

System developers require a very deep understanding of Java. At least three years of Java coding experience in a system environment is recommended. A Framework implementation will use areas of Java that are not normally encountered in traditional applications. Detailed understanding is required of class loaders, garbage collection, Java 2 security, and Java native library loading.

Architects should focus on the introduction of each subject. This introduction contains a general overview of the subject, the requirements that influenced its design, and a short description of its operation as well as the entities that are used. The introductory sections require knowledge of Java concepts like classes and interfaces, but should not require coding experience.

Most of these specifications are equally applicable to application developers and system developers.

1.2 Version Information

Each OSGi specification has an independent version, and packages defined by those specifications have their own versions as well. Specification and package versions follow semantic versioning rules.

1.3 References

- [1] *OSGi Specifications*
<https://docs.osgi.org/specification/>

DRAFT

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

137 REST Management Service Specification

Version 1.0

137.1 Introduction

Cloud computing is a continuing trend in the IT industry. Due to its service model which embraces dynamism as opposed to masking it, OSGi appears to be an ideal base for building scalable and dependable applications for the cloud where changes in the deployment, network topology, and service availability are the norm rather than the exception. One of the possible scenarios for OSGi to be successfully applied to cloud computing is using it in a Platform as a Service (PaaS) spirit. Users write their bundles and can deploy them to a provided OSGi instance running in the cloud. This, however, requires the platform provider to expose the OSGi management API to the end user and make them available through a network protocol. One of the popular approaches in cloud computing to remote communication is the use of RESTful web services.

Representational State Transfer (REST) is the architectural style of the world wide web. It can be described as a set of constraints that govern the interactions between the main components of the Internet. Recently, REST style interaction has gained popularity as a architecture for web services (RESTful web services), mainly to overcome the perceived complexity and verbosity of SOAP-based web services. This specification describes a REST interface for framework management, client-side Java and JavaScript APIs, and an extension mechanism through which other bundles can contribute their own RESTful management APIs and make them discoverable by clients.

137.1.1 Essentials

- *Client-Server* - A separation of concern between the entity responsible for the user-interaction (client) and the other entity (server) responsible for data storage. For instance, in the original world wide web the browser is the client rendering and presenting the content delivered by one or more web servers. As a result, web content becomes more portable and content providers more scalable.
- *Stateless* - State is entirely kept at the client side. Therefore, every request must contain all state required for the server to accomplish the transaction and deliver content. The main rationale behind this design constraint is to again improve the scalability since in a pure stateless design the server resources are not burdened with maintaining any client state. Another perceived advantage is that the failure models of stateless interactions is simpler and fault tolerance easier to achieve.
- *Cacheable* - Content marked as cacheable can be temporarily stored and used to immediately answer future equivalent requests and improve efficiency and reduce network utilization and access latencies. Due to the end-to-end principle, caches can be placed where necessary, e.g., at the client (forward-proxy), at the server side (backward-proxy), or somewhere in-between for example in a content delivery network. Content marked as non-cacheable must be freshly retrieved with every request even in the presence of caches.
- *Layered* - Layering introduces natural boundaries to coupling since every layer only accesses the services provided by the lower layer and provides services to the next higher layer.

Management	Service	Interacting with the REST Management Service Specification	REST Version	1.0
		• <i>Uniform Interface</i> - Generality of component interfaces provides a natural decoupling of implementation and interface. REST furthermore encourages the separation of identifiable resources (addressing) and their representation (content delivery).		

137.1.2 Entities

- *Resource* - A resource is an abstract piece of information that can be addressed by a resource identifier. The mapping of a resource to a concrete set of entities can vary over time.
- *Representation* - A representation is a sequence of bytes plus associated meta-data that describe the state of a resource. The data format of a representation is called the media-type. Every concrete representation of a resource is just one of arbitrarily many possible representations. The selection of a concrete representation of a resource can be made according to the media types supported by both the client and the server.
- *REST Management Service* - The management service exposes a REST API for remotely managing an OSGi framework through the network in a lightweight and portable fashion.
- *Client* - The client is a machine using the management service by issuing REST requests through the network. It can do so either directly or indirectly, i.e., through client-side libraries using the REST calls internally.

137.1.3 Synopsis

The manageable entities of an OSGi framework are mapped to resources accessible through resource identifiers. These identifiers are relative to the (usually externally accessible) root URL of the management service. Clients can either discover this root URL or receive it through configuration. Subsequently, a client is able to introspect the state of the framework and perform management operations.

The internal state of a framework resource is expressed and transmitted as a representation. The format of the representation is subject to a mutual agreement between client and management service regarding media types commonly supported by both endpoints. This specification describes two representation formats: JSON and XML.

137.2 Interacting with the REST Management Service

The REST Management Service is not a traditional OSGi service and it does not appear in the service registry. Its purpose is to expose a management interface to clients which can perform operations on the framework through a network connection. Therefore, it is ideally suited for situations where the user of an OSGi framework does not have direct access to the machine it is running on, a typical situation in Infrastructure as a Service (IaaS) or Platform as a Service (PaaS). However, even in other domains having a lightweight and easily accessible management solution can be of benefit, e.g., for embedded devices. The advantage of REST is that it uses HTTP and therefore does usually not interfere with firewalls. Furthermore, the REST format is easily embeddable into client-side scripting technologies like JavaScript and can be consumed in web browsers.

Much of the value of the REST Management Service lies in client-side libraries which can use the REST protocol and interact with the OSGi framework through the Management Service. Therefore, this specification contains API for two clients, a Java Client API and a JavaScript Client API.

137.2.1 Resource Identifier Overview

The REST Management Service comprises of a set of resources that can be retrieved and in some cases also modified through REST requests. These resources need to be made available under well-defined paths so that clients can interact with them. As the initial entry point a client receives a URL to the REST Management Service. This can be done, e.g., as part of the creation of a cloud-based OSGi

framework, and the precise mechanism would be proprietary to the cloud platform used. Relative to this URL the client can access the resources through the following resource identifiers:

```
framework
framework/state
framework/startlevel
framework/bundles
```

```
framework/bundles/representations
```

```
framework/bundle/{bundleid}
framework/bundle/{bundleid}/state
framework/bundle/{bundleid}/startlevel
framework/bundle/{bundleid}/header
framework/services
```

```
framework/services/representations
```

```
framework/service/{serviceid}
```

framework/bundle/o/state is an alias for framework/state

Extensions to the REST Management Service can be discovered by visiting the Extensions Resource at:

```
extensions
```

For more details on the extension mechanism see *Extending the REST Management Service* on page 21

137.2.2

Filtering Results

The bundles, bundles/representations, services, and services/representations resources allow the use of a query parameter which specifies a filter to restrict the result set. The filter expression follows the Core Specifications *Framework Filter Syntax*; see [1] *Framework Filter Syntax* .

Filters on services are matched against the service attributes. The query parameter is of the form:

```
framework/services?filter=ldap-filter
```

Filters on bundles are matched against the attributes of capabilities in the respective namespaces. Filters on bundles have the form:

```
framework/bundles?namespace1=ldap-filter1&namespace2=ldap-filter2&...
```

If multiple capabilities for a given namespace are present, then a filter succeeds when one of these capabilities matches. When multiple filter expressions across namespaces are given, these are combined with the *and* operator.

137.2.3

Content Type Matching

Resources can present themselves through different representation variants. An implementation of this specification must support at least the JSON representation and the XML representation of resources. Clients can support a subset of representations. Matching the clients capabilities to understand certain representation formats with the servers supported formats follows the typical HTTP pattern of content negotiation and requires the client to set corresponding HTTP Accept headers for supported formats in the form of their media types. This specification describes the format and media types for representations in JSON and XML format in *Representations* on page 15 .

Implementations of the REST Management Service offering different variants of representations must return the best matching variant based on the HTTP accept header. In addition, they must re-

spect the file extensions defined for the different media types as specified in the respective IETF RFC (e.g., ".xml" as specified in IETF RFC 3032 and ".json" as specified in IETF RFC 4627). If a file extension is appended to the resource, an implementation must return the variant mandated by the file extension provided that it supports this content type.

137.3 Resources

The framework and its state is mapped to a set of different resources. Each resource is accessible through a resource identifier, as summarized in *Resource Identifier Overview* on page 8 .

137.3.1 Framework Startlevel Resource

framework/startlevel

The startlevel resource represents the active start level of the framework. It supports the GET and PUT requests.

137.3.1.1 GET

The GET request retrieves a *Framework Startlevel Representation* from the REST management service. The request can return the following status codes:

- 200 (OK): the request has been served successfully and the body of the response is a startlevel representation.
- 406 (NOT ACCEPTABLE): the REST management service does not support any of the requested representations.

137.3.1.2 PUT

The PUT request sets the target framework startlevel. The body of the request needs to be a *Framework Startlevel Representation* . The request can return the following status codes:

- 204 (NO CONTENT): the request was received and valid. The framework will asynchronously start to adjust the framework startlevel until the target startlevel has been reached.
- 415 (UNSUPPORTED MEDIA TYPE): the request had a media type that is not supported by the REST management service.
- 400 (BAD REQUEST): the REST management service received an `IllegalArgumentException` when trying to adjust the framework startlevel, e.g., because the requested startlevel was zero or negative.

137.3.2 Bundles Resource

framework/bundles

The bundles resource represents the list of all bundles installed on the managed framework. It supports the GET request and two syntactically different forms of POST requests which are used to install new bundles to the framework.

Results for this resource can be filtered as described in *Filtering Results* on page 9 .

137.3.2.1 GET

The GET request retrieves a *Bundle List Representation* from the REST management service. The request can return the following status codes:

- 200 (OK): the request has been served successfully and the body of the response is a bundle list representation.
- 406 (NOT ACCEPTABLE): the REST management service does not support any of the requested representations.

137.3.2.2**POST with Location String**

The POST request installs a new bundle to the managed framework and thereby logically appends it to the bundles resource. The new bundle to be installed is referenced by a location string which is passed as the body of the request. In order to disambiguate the request from the other form of POST, the content type must be set to text/plain. In practice, the location string is usually a URL. Since the framework will use the location retrieving the physical bundle, it needs to be accessible from the remotely managed framework and not necessarily from the managing client.

The management service implementation must check if the result of the install request matches the requested bundle since the OSGi framework will return an existing bundle object as the return value of an install call if there was already one with the same location string installed. One way of doing it is comparing the last modification timestamp. A detected collision is indicated to the requesting clients through an error code 409.

The body of the response is a *Bundle Representation* of the newly installed bundle. The following status codes can be returned:

- 200 (OK): the bundle has been successfully installed and the body of the response contains a *Bundle Representation* .
- 400 (BAD REQUEST): the REST management service received a *BundleException* when trying to install. The body of the message is a *Bundle Exception Representation* describing the reason why the installation did not succeed.
- 409 (CONFLICT): there is already a bundle installed with the same location string.

137.3.2.3**POST with Bundle**

This variant of the POST request uploads the bundle as the body of the request. The media type of the request should be set to application/vnd.osgi.bundle which must be supported by all REST management services. Implementations are free to accept other media types for this request with the exception of text/plain. For instance, they can opt to additionally support application/zip or application/x-jar.

Clients should use the HTTP Content-Location field to set a bundle location. If no content location is given, REST management service implementations must generate a unique location string in order to avoid unintended collisions between unrelated bundles.

The body of the response is *Bundle Representation* of the newly installed bundle. The following status codes can be returned:

- 200 (OK): the bundle has been successfully installed and the body of the response contains the URI.
- 400 (BAD REQUEST): the REST management service received a *BundleException* when trying to install. The body of the message is a *Bundle Exception Representation* describing the reason why the installation did not succeed.
- 409 (CONFLICT): there is already a bundle installed with the same location string.

137.3.3**Bundles Representations Resource**

framework/bundles/representations

137.3.3.1**GET of the Representations**

The bundles resource returns a list of the URIs of all bundles installed on the framework. For clients interested in the details of multiple bundles there is also the possibility to retrieve the bundle representation of each installed bundle with a single request through the *bundles/representations* resource.

The body of the response is a *Bundle Representations List Representation* . The request can return the following status codes:

Results for this resource can be filtered as described in *Filtering Results* on page 9 .

- 200 (OK): the request has been served successfully and the body of the response is a bundle list representation.
- 406 (NOT ACCEPTABLE): the REST management service does not support any of the requested representations.

137.3.4 Bundle Resource

framework/bundle/{bundleid}

The bundle resource represents a single, distinct bundle in the system. Hence, it has to be qualified by a bundle id. The resource supports the GET, two variants of PUT, and the DELETE requests.

137.3.4.1 GET

The GET request retrieves a *Bundle Representation* from the REST management service. The request can return the following status codes:

- 200 (OK): the request has been served successfully and the body of the response is a bundle representation.
- 404 (NOT FOUND): there is not bundle with the given bundle id.
- 406 (NOT ACCEPTABLE): the REST management service does not support any of the requested representations.

137.3.4.2 PUT with Location String

The PUT request updates the bundle with a new version, referenced by a location string which is passed as the body of the request. In order to disambiguate the request from the other form of PUT, the content type must be set to text/plain. The same rationale applies as for *POST with Location String* and *POST with Bundle* on page 11, if a location string is given it must point to a location reachable by the managed framework. If no location string is passed as the body of the request, the framework will perform an update based on the existing bundle's location string.

The body of the response is *Bundle Representation* of the updated bundle. The following status codes can be returned:

- 204 (NO CONTENT): the request was received and valid and the framework has issued the update.
- 400 (BAD REQUEST): the REST management service received a *BundleException* when trying to update. The body of the message is a *Bundle Exception Representation* describing the reason why the update did not succeed.
- 404 (NOT FOUND): there is not bundle with the given bundle id.

137.3.4.3 PUT with Bundle

The PUT request updates the bundle with a new version, uploaded as the body of the request. The media type of the request should be set to application/vnd.osgi.bundle which must be supported by all REST management services. Implementations are free to accept other media types for this request with the exception of text/plain. For instance, they can opt to additionally support application/zip or application/x-jar.

The body of the response is *Bundle Representation* of the updated bundle. The following status codes can be returned:

- 204 (NO CONTENT): the request was received and valid and the framework has issued the update.
- 400 (BAD REQUEST): the REST management service received a *BundleException* when trying to update. The body of the message is a *Bundle Exception Representation* describing the reason why the update did not succeed.
- 404 (NOT FOUND): there is not bundle with the given bundle id.

137.3.4.4**DELETE**

The DELETE request uninstalls the bundle from the framework.

The body of the response is *Bundle Representation* of the uninstalled bundle, where the bundle state will be UNINSTALLED. The following status codes can be returned:

- 204 (NO CONTENT): the request was received and valid and the framework has uninstalled the bundle.
- 400 (BAD REQUEST): the REST management service received a *BundleException* when trying to uninstall. The body of the message is a *Bundle Exception Representation* describing the reason why the uninstallation did not succeed.
- 404 (NOT FOUND): there is not bundle with the given bundle id.

137.3.5**Bundle State Resource**

framework/bundle/{bundleid}/state

The bundle state resource represents the internal state of an installed bundle qualified through its bundle id. It supports the GET and PUT requests.

137.3.5.1**GET**

The GET request retrieves a *Bundle State Representation* from the REST management service. The request can return the following status codes:

- 200 (OK): the request has been served successfully and the body of the response is a bundle state representation.
- 404 (NOT FOUND): there is not bundle with the given bundle id.
- 406 (NOT ACCEPTABLE): the REST management service does not support any of the requested representations.

137.3.5.2**PUT**

The PUT request sets the target state for the given bundle. This can, e.g., be state=32 for transitioning the bundle to started, or state=4 for stopping the bundle and transitioning it to resolved. The body of the request needs to be a *Bundle State Representation*. Not all state transitions are valid. The body of the response is the new *Bundle State Representation*. The request can return the following status codes:

- 200 (OK): the request was received and valid. The framework has performed a state change and the new bundle state is contained in the body.
- 400 (BAD REQUEST): the REST management service received a *BundleException* when trying to perform the state transition. The body of the message is a *Bundle Exception Representation* describing the reason why the operation did not succeed.
- 402 (PRECONDITION FAILED): the requested target state is not reachable from the current bundle state or is not a target state. An example such state is the STOPPING state.
- 404 (NOT FOUND): there is not bundle with the given bundle id.
- 415 (UNSUPPORTED MEDIA TYPE): the request had a media type that is not supported by the REST management service.

137.3.6**Bundle Header Resource**

framework/bundle/{bundleid}/header

The bundle header resource represents manifest header of a bundle which is qualified by its bundle id. It can only be read through a GET request.

137.3.6.1**GET**

The GET request retrieves a *Bundle Header Representation* from the REST management service. The raw header value is used unless an Accept-Language header is set on the HTTP request. If multiple accepted languages are set only the first is used to localize the header. The request can return the following status codes:

- 200 (OK): the request has been served successfully and the body of the response is a bundle header representation.
- 404 (NOT FOUND): there is not bundle with the given bundle id.
- 406 (NOT ACCEPTABLE): the REST management service does not support any of the requested representations.

137.3.7**Bundle Startlevel Resource**

framework/bundle/{bundleid}/startlevel

The bundle startlevel resource represents the start level of the bundle qualified by its bundle id. It supports the GET and PUT requests.

137.3.7.1**GET**

The GET request retrieves a *Bundle Startlevel Representation* from the REST management service. The request can return the following status codes:

- 200 (OK): the request has been served successfully and the body of the response is a bundle startlevel representation.
- 404 (NOT FOUND): there is not bundle with the given bundle id.
- 406 (NOT ACCEPTABLE): the REST management service does not support any of the requested representations.

137.3.7.2**PUT**

The PUT request sets the target bundle startlevel. The body of the request needs to be a *Bundle Startlevel Representation*, however only the startlevel property is used. The request can return the following status codes:

- 200 (OK): the request was received and valid. The REST management service has changed the bundle startlevel according to the target value. The body of the response is the new bundle startlevel representation.
- 400 (BAD REQUEST): either the target startlevel state involved invalid values, e.g., a startlevel smaller or equal to zero and the REST management service got an *IllegalArgumentException*, or the REST management service received a *BundleException* when trying to perform the startlevel change. In the latter case, the body of the message is a *Bundle Exception Representation* describing the reason why the operation did not succeed.
- 404 (NOT FOUND): there is not bundle with the given bundle id.
- 415 (UNSUPPORTED MEDIA TYPE): the request had a media type that is not supported by the REST management service.

137.3.8**Services Resource**

framework/services

The services resource represents the set of all services available on the framework, optionally constrained by a filter expression. It is read-only and therefore only supports the GET request.

Results for this resource can be filtered as described in *Filtering Results* on page 9 .

137.3.8.1**GET**

The GET request retrieves a *Service List Representation* from the REST management service. The request can return the following status codes:

- 200 (OK): the request has been served successfully and the body of the response is a service list representation.
- 400 (BAD REQUEST): the provided filter expression was not valid.
- 406 (NOT ACCEPTABLE): the REST management service does not support any of the requested representations.

137.3.9**Services Representations Resource**

framework/services/representations

137.3.9.1**GET of the Representations**

The services resource returns a list of the URIs of all services registered on the framework. For clients interested in the details of multiple services there is also the possibility to retrieve the service representation of each available service with a single request through the *services/representations* resource. The body of the response is a *Service Representations List Representation* from the REST management service. The request can return the following status codes:

Results for this resource can be filtered as described in *Filtering Results* on page 9 .

- 200 (OK): the request has been served successfully and the body of the response is a service list representation.
- 400 (BAD REQUEST): the provided filter expression was not valid.
- 406 (NOT ACCEPTABLE): the REST management service does not support any of the requested representations.

137.3.10**Service Resource**

framework/service/{serviceid}

The service resource represents a single, distinct service in the framework. Hence, it has to be qualified by a service id. Services can only be read through the REST Management Service and therefore only support the GET request.

137.3.10.1**GET**

The GET request retrieves a *Service Representation* . The request can return the following status codes:

- 200 (OK): the request has been served successfully and the body of the response is a service representation.
- 404 (NOT FOUND): there is not service with the given service id.
- 406 (NOT ACCEPTABLE): the REST management service does not support any of the requested representations.

137.4 Representations**137.4.1****Bundle Representation****137.4.1.1****JSON**

Content-Type: application/org.osgi.bundle+json

```
{
  "id":0,
  "lastModified":1314999275542,
  "state":32,
  "symbolicName":"org.eclipse.osgi",
  "version":"3.7.0.v20110613"
}
```

137.4.1.2**XML**

Content-Type: application/org.osgi.bundle+xml

```
<bundle>
  <id>0</id>
  <lastModified>1314999275542</lastModified>
  <state>32</state>
  <symbolicName>org.eclipse.osgi</symbolicName>
  <version>3.7.0.v20110613</version>
</bundle>
```

137.4.2**Bundles Representations****137.4.2.1****Bundle List Representation****137.4.2.1.1****JSON**

Content-Type: application/org.osgi.bundles+json

[bundleURI, bundleURI, ..., bundleURI]

137.4.2.1.2**XML**

Content-Type: application/org.osgi.bundles+xml

```
<bundles>
  <uri>bundleURI</uri>
  <uri>bundleURI</uri>
  ...
  <uri>bundleURI</uri>
</bundles>
```

137.4.2.2**Bundle Representations List Representation****137.4.2.2.1****JSON**

Content-Type: application/org.osgi.bundles.representations+json

[BUNDLE REPRESENTATION, BUNDLE REPRESENTATION, ..., BUNDLE REPRESENTATION]

137.4.2.2.2**XML**

Content-Type: application/org.osgi.bundles.representations+xml

```
<bundles>
  BUNDLE REPRESENTATION
  BUNDLE REPRESENTATION
  ...
  BUNDLE REPRESENTATION
</bundles>
```

137.4.3 Bundle State Representation

137.4.3.1 JSON

Content-Type: application/org.osgi.bundlestate+json

```
{
  "state": 32
  "options": 1
}
```

The options are used in start or stop calls. Valid options include, e.g., Bundle.START_TRANSIENT and Bundle.START_ACTIVATION_POLICY.

137.4.3.2 XML

Content-Type: application/org.osgi.bundlestate+xml

```
<bundleState>
  <state>32</state>
  <options>1</options>
</bundleState>
```

137.4.4 Bundle Header Representation

137.4.4.1 JSON

Content-Type: application/org.osgi.bundleheader+json

```
{
  key: value,
  key: value,
  ...
  key: value
}
```

137.4.4.2 XML

Content-Type: application/org.osgi.bundleheader+xml

```
<bundleHeader>
  <entry key="key" value="value"/>
  <entry key="key" value="value"/>
  ...
  <entry key="key" value="value"/>
</bundleHeader>
```

137.4.5 Framework Startlevel Representation

137.4.5.1 JSON

Content-Type: application/org.osgi.frameworkstartlevel+json

```
{
  "startLevel": 6,
  "initialBundleStartLevel": 4
}
```

137.4.5.2**XML**

Content-Type: application/org.osgi.framework.startlevel+xml

```
<frameworkStartLevel>
  <startLevel>6</startLevel>
  <initialBundleStartLevel>4</initialBundleStartLevel>
</frameworkStartLevel>
```

137.4.6**Bundle Startlevel Representation****137.4.6.1****JSON**

Content-Type: application/org.osgi.bundle.startlevel+json

```
{
  "startLevel": 6
  "activationPolicyUsed": true
  "persistentlyStarted": false
}
```

137.4.6.2**XML**

Content-Type: application/org.osgi.bundle.startlevel+xml

```
<bundleStartLevel>
  <startLevel>6</startLevel>

  <activationPolicyUsed>true</activationPolicyUsed>
  <persistentlyStarted>false</persistentlyStarted>
</bundleStartLevel>
```

137.4.7**Service Representation****137.4.7.1****JSON**

Content-Type: application/org.osgi.service+json

```
{
  "id": 10,
  "properties": {
    "prop1": "val1",
    "prop2": 2.82,
    ...
    "prop3": true
  },
  "bundle": bundleURI,
  "usingBundles": [bundleURI, bundleURI, ... bundleURI]
}
```

Note: service properties are converted to JSON-supported data types where possible: "string", number or boolean (true/false). If there is no conversion to JSON data types is possible the toString() result is used as a string value.

137.4.7.2**XML**

Content-Type: application/org.osgi.service+xml

```

<service>
  <id>10</id>
  <properties>
    <property name="prop1" value="val1"/>
    <property name="prop2" type="Float" value="2.82"/>
    ...
    <property name="prop3" type="Boolean" value="true"/>
  </properties>
  <bundle>bundleURI</bundle>
  <usingBundles>
    <bundle>bundleURI</bundle>
    <bundle>bundleURI</bundle>
    ...
    <bundle>bundleURI</bundle>
  </usingBundles>
</service>

```

Note: service properties are represented using the same method as used for the property XML element in the Declarative Services specification, see ???. Service properties that cannot be represented using the supported data types, will be represented as String values obtained via the toString() method.

137.4.8 Services Representations

137.4.8.1 Service List Representation

137.4.8.1.1 JSON

Content-Type: application/org.osgi.services+json

[serviceURI, serviceURI, ..., serviceURI]

137.4.8.1.2 XML

Content-Type: application/org.osgi.services+xml

```

<services>
  <uri>serviceURI</uri>
  <uri>serviceURI</uri>
  ...
  <uri>serviceURI</uri>
</services>

```

137.4.8.2 Service Representations List Representation

137.4.8.2.1 JSON

Content-Type: org.osgi.services.representations+json

[SERVICE REPRESENTATION, SERVICE REPRESENTATION, ..., SERVICE REPRESENTATION]

137.4.8.2.2 XML

Content-Type: application/org.osgi.services.representations+xml

```

<services>
  SERVICE REPRESENTATION
  SERVICE REPRESENTATION

```

```
...
  SERVICE REPRESENTATION
</services>
```

137.4.9 Bundle Exception Representation

137.4.9.1 JSON

Content-Type: application/org.osgi.bundleexception+json

```
{
  "typecode": 5,
  "message": "BundleException: Bundle activation error"
}
```

137.4.9.2 XML

Content-Type: application/org.osgi.bundleexception+xml

```
<bundleexception>
  <typecode>5</typecode>
  <message>BundleException: Bundle activation error</message>
</bundleexception>
```

137.5 Clients

The REST service can be used by a variety of clients directly. In addition this specification describes Client APIs built over this REST protocol to facilitate use from Java and JavaScript clients.

137.5.1 Java Client

The Java Client provides a Java API over the REST API providing a convenient and portable way to use this API from a Java application.

To use the Java Client, obtain the [RestClientFactory](#) service. Create a client by providing the root URL of the REST service, for example:

```
RestClientFactory restClientFactory = ... // from Service Registry
RestClient restClient = restClientFactory.createRestClient(
    new URI("http://localhost:8080/restendpoint"));
```

```
// Now we can start interacting
Collection<String> bundles = restClient.getBundlePaths();
BundleDTO newBundle = restClient.installBundle(bundleLocation, bundleStream);
restClient.startBundle(newBundle.id);
```

The more details on the Java Client can be found in the [org.osgi.service.rest.client](#) API documentation section.

137.5.2 JavaScript Client

This specification also describes a JavaScript client to the REST Management service. This client makes it easy to manage an OSGi framework from any JavaScript environment, including Web Browsers.

The JavaScript client follows the *promises* programming style; the request is made asynchronously and a `success()` or `failure()` callback is made when the response arrives.

To use the JavaScript client create an instance of `OSGiRestClient` providing the root URL of the REST service.

```
var client = new OSGiRestClient('http://localhost:8080/restendpoint');
client.installBundle({
  success : function(res) {
    // Start the bundle once the install has finished
    client.startBundle(res.id);
  },
  failure : function(httpCode, res) {
    // handle failure
  }
});
```

More details on the JavaScript Client can be found in the [JavaScript Client API](#) API documentation section.

137.6 Extending the REST Management Service

This specification describes a REST-based management interface for Core Framework functionality. Other services in the framework might also benefit from management access through REST. This can involve services specified by the OSGi Working Group as part of the Core or Compendium Specifications but also application-specific functionality provided by the developer. It is desirable to expose such management services as extensions of the REST Management Service.

This REST service can be implemented by using various technologies such as Java Servlets, Restlet, JAX-RS, and others. Therefore, it might not always be possible to integrate extensions at the implementation level because they might use other underlying technologies to implement their REST interface. Defining a format for delegating requests between the REST Management Service and extensions would furthermore necessarily expose implementation details and is therefore not feasible either. As a consequence, this specification only describes how to logically integrate extensions with the REST Management Service. Implementations of this specification might offer mechanisms for tighter integration for the case that extensions are developed using the same underlying technology.

The main purpose of the extension mechanism is to advertise extensions to the core REST implementation, which makes them discoverable for clients. This mechanism can be used to check if a REST interface exists for a specific service. This is done through the *Extensions Resource* which contains a description and a path for every extension currently available. Implementations that want to contribute their extensions to the REST Management Service can do so by registering the `RestApiExtension` service using the [4] *Whiteboard Pattern*. The extension interface is only a marker and the relevant information is exposed through the `NAME`, `URI_PATH` and optionally `SERVICE` properties. Note that it is the responsibility of the extension to ensure that the endpoint announced via the `RestApiExtension` service is actually present. The Whiteboard service does not realize the extension endpoint; it purely announces it to the main REST implementation for inclusion in the *Extensions Resource*.

In order to be discoverable REST interface extensions to OSGi Core or Compendium services must use their canonical package name as advertised name. E.g., the name of the REST interface for the User Admin must be `org.osgi.service.useradmin`. This way, a client is able to check if there is a given extension available on a host. User-defined extensions should use the package name of the service they provide management capabilities for.

137.6.1 Extensions Resource

extensions

The extensions resource enumerates all extensions currently registered through the Whiteboard Pattern. It is read-only and therefore only supports the GET request.

137.6.1.1 GET

The GET request retrieves a *Extensions Representation*. The request can return the following status codes:

- 200 (OK): the request has been served successfully and the body of the response is a extension list representation.
- 406 (NOT ACCEPTABLE): the REST management service does not support any of the requested representations.

137.6.2 Extensions Representation

137.6.2.1 JSON

Content-Type: application/org.osgi.extensions+json

```
[ { "name" : "org.osgi.service.event", "path" : "contributions/eventadmin",
    "service" : 12 }, ... ]
```

137.6.2.2 XML

Content-Type: application/org.osgi.extensions+xml

```
<extensions>
  <extension>
    <name>org.osgi.service.event</name>
    <path>contributions/eventadmin</path>
    <service>12</service>
  </extension>
</extensions>
```

137.7 XML Schema

The namespace for XML representations is:

<http://www.osgi.org/xmlns/rest/v1.0.0>

The recommended prefix for this namespace is rest.

```
<?xml version="1.0" encoding="UTF-8"?>
<!--
Copyright (c) Contributors to the Eclipse Foundation

Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at

    http://www.apache.org/licenses/LICENSE-2.0

Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.

SPDX-License-Identifier: Apache-2.0
-->

<schema xmlns="http://www.w3.org/2001/XMLSchema"
  xmlns:rest="http://www.osgi.org/xmlns/rest/v1.0.0"
  targetNamespace="http://www.osgi.org/xmlns/rest/v1.0.0"
```

```

elementFormDefault="unqualified"
attributeFormDefault="unqualified"
version="1.0.0">

<annotation>
  <documentation xml:lang="en">
    This is the XML Schema for
    XML representations used by
    the REST Management Service
    Specification.
  </documentation>
</annotation>

<element name="bundle" type="rest:Tbundle">
  <annotation>
    <documentation xml:lang="en">
      Representation for the
      application/org.osgi.bundle+xml content type.
    </documentation>
  </annotation>
</element>
<complexType name="Tbundle">
  <all>
    <element name="id" type="long" />
    <element name="lastModified" type="long" />
    <element name="state" type="integer" />
    <element name="symbolicName" type="string" />
    <element name="version" type="string" />
  </all>
</complexType>

<element name="bundles" type="rest:Tbundles">
  <annotation>
    <documentation xml:lang="en">
      Representation for the
      application/org.osgi.bundles+xml and
      application/org.osgi.bundles.representations+xml content
      types.
    </documentation>
  </annotation>
</element>
<complexType name="Tbundles">
  <choice>
    <element name="uri" type="string" minOccurs="0"
      maxOccurs="unbounded">
      <annotation>
        <documentation xml:lang="en">
          Representation for the
          application/org.osgi.bundles+xml content type.
        </documentation>
      </annotation>
    </element>
    <element name="bundle" type="rest:Tbundle"
      minOccurs="0" maxOccurs="unbounded">
      <annotation>
        <documentation xml:lang="en">
          Representation for the
          application/org.osgi.bundles.representations+xml
          content type.
        </documentation>
      </annotation>
    </element>
  </choice>
</complexType>

<element name="bundleState" type="rest:TbundleState">
  <annotation>
    <documentation xml:lang="en">
      Representation for the
      application/org.osgi.bundlestate+xml content type.
    </documentation>
  </annotation>
</element>
<complexType name="TbundleState">
  <all>

```

```

    <element name="state" type="integer" />
    <element name="options" type="integer" />
  </all>
</complexType>

<element name="bundleHeader" type="rest:TbundleHeader">
  <annotation>
    <documentation xml:lang="en">
      Representation for the
      application/org.osgi.bundleheader+xml content type.
    </documentation>
  </annotation>
</element>
<complexType name="TbundleHeader">
  <sequence>
    <element name="entry" minOccurs="0" maxOccurs="unbounded">
      <complexType>
        <attribute name="key" type="string" use="required" />
        <attribute name="value" type="string" use="required" />
      </complexType>
    </element>
  </sequence>
</complexType>

<element name="frameworkStartLevel" type="rest:TframeworkStartLevel">
  <annotation>
    <documentation xml:lang="en">
      Representation for the
      application/org.osgi.frameworkstartlevel+xml content
      type.
    </documentation>
  </annotation>
</element>
<complexType name="TframeworkStartLevel">
  <all>
    <element name="startLevel" type="integer" />
    <element name="initialBundleStartLevel" type="integer" />
  </all>
</complexType>

<element name="bundleStartLevel" type="rest:TbundleStartLevel">
  <annotation>
    <documentation xml:lang="en">
      Representation for the
      application/org.osgi.bundlestartlevel+xml content type.
    </documentation>
  </annotation>
</element>
<complexType name="TbundleStartLevel">
  <all>
    <element name="startLevel" type="integer" />
    <element name="activationPolicyUsed" type="boolean" />
    <element name="persistentlyStarted" type="boolean" />
  </all>
</complexType>

<element name="service" type="rest:Tservice">
  <annotation>
    <documentation xml:lang="en">
      Representation for the
      application/org.osgi.service+xml content type.
    </documentation>
  </annotation>
</element>
<complexType name="Tservice">
  <all>
    <element name="id" type="long" />
    <element name="properties">
      <complexType>
        <sequence>
          <element name="property" minOccurs="0"
            maxOccurs="unbounded">
            <complexType>
              <simpleContent>
                <extension base="string">

```

```

        <attribute name="name"
            type="string" use="required" />
        <attribute name="value"
            type="string" use="optional" />
        <attribute name="type"
            default="String" use="optional">
            <simpleType>
                <restriction
                    base="string">
                    <enumeration
                        value="String" />
                    <enumeration
                        value="Long" />
                    <enumeration
                        value="Double" />
                    <enumeration
                        value="Float" />
                    <enumeration
                        value="Integer" />
                    <enumeration
                        value="Byte" />
                    <enumeration
                        value="Character" />
                    <enumeration
                        value="Boolean" />
                    <enumeration
                        value="Short" />
                </restriction>
            </simpleType>
        </attribute>
    </extension>
</simpleContent>
</complexType>
</element>
</sequence>
</complexType>
</element>
<element name="bundle" type="string" />
<element name="usingBundles">
    <complexType>
        <sequence>
            <element name="bundle" type="string"
                minOccurs="0" maxOccurs="unbounded" />
        </sequence>
    </complexType>
</element>
</all>
</complexType>
<element name="services" type="rest:Tservices">
    <annotation>
        <documentation xml:lang="en">
            Representation for the
            application/org.osgi.services+xml and
            application/org.osgi.services.representations+xml
            content types.
        </documentation>
    </annotation>
</element>
<complexType name="Tservices">
    <choice>
        <element name="uri" type="string" minOccurs="0"
            maxOccurs="unbounded">
            <annotation>
                <documentation xml:lang="en">
                    Representation for the
                    application/org.osgi.services+xml content type.
                </documentation>
            </annotation>
        </element>
        <element name="service" type="rest:Tservice"
            minOccurs="0" maxOccurs="unbounded">
            <annotation>
                <documentation xml:lang="en">
                    Representation for the

```

```

        application/org.osgi.services.representations+xml
        content type.
      </documentation>
    </annotation>
  </element>
</choice>
</complexType>

<element name="bundleexception" type="rest:Tbundleexception">
  <annotation>
    <documentation xml:lang="en">
      Representation for the
      application/org.osgi.bundleexception+xml content type.
    </documentation>
  </annotation>
</element>
<complexType name="Tbundleexception">
  <all>
    <element name="typecode" type="integer" />
    <element name="message" type="string" />
  </all>
</complexType>

<element name="extensions" type="rest:Textensions">
  <annotation>
    <documentation xml:lang="en">
      Representation for the
      application/org.osgi.extensions+xml content type.
    </documentation>
  </annotation>
</element>
<complexType name="Textensions">
  <sequence>
    <element name="extension" minOccurs="0" maxOccurs="unbounded">
      <complexType>
        <all>
          <element name="name" type="string" />
          <element name="path" type="string" />
          <element name="service" type="long" minOccurs="0" />
        </all>
      </complexType>
    </element>
  </sequence>
</complexType>
</schema>

```

The schema is also available in digital form from [3] *OSGi XML Schemas* .

137.8 Capabilities

137.8.1 osgi.implementation Capability

An implementation of this specification must provide the `osgi.implementation` capability with name `osgi.rest`. This capability can be used by provisioning tools and during resolution to ensure that a REST Management implementation is present to handle REST requests defined in this specification. The capability must also declare a uses constraint on the `org.osgi.service.rest` package:

```

Provide-Capability: osgi.implementation;
  osgi.implementation="osgi.rest";
  uses="org.osgi.service.rest";
  version:Version="

```

1.0

"

This capability must follow the rules defined for the ???.

137.8.2 **osgi.service Capability**

A bundle providing the [RestClientFactory](#) service as described by this specification must inform tools about this service by providing the `osgi.service` capability representing this service. This capability must also declare a uses constraint for the `org.osgi.service.rest.client` package:

```
Provide-Capability: osgi.service;  
    objectClass:List<String>="org.osgi.service.rest.client.RestClientFactory";  
    uses:="org.osgi.service.rest.client"
```

This capability must follow the rules defined for the ???.

137.9 **Security**

Like any externally visible management interface, the REST interface exposes privileged operations and hence requires access control. Since REST builds upon the HTTP(s) protocol, authentication mechanisms and encryption can be applied the same way as usually done for web servers: they can be layered below the REST protocol. E.g., confidentiality of the transmitted commands can be ensured by using HTTPS as the underlying transport. Authentication can be added by requiring, e.g., basic authentication prior to accepting a REST command. The REST interface should only be implemented by a trusted bundle. Implementations of this specification require all Admin Permissions and all Service Permissions.

137.10 **org.osgi.service.rest**

Rest Service Package Version 1.0.

137.10.1 **Summary**

- `RestApiExtension` - Marker interface for registering extensions to the Rest API service.

137.10.2 **public abstract RestApiExtension**

Marker interface for registering extensions to the Rest API service.

The REST service provides a RESTful interface to clients that need to manage an OSGi framework through a network connection. Other components running on the same framework can contribute their own specific REST interface and make it available and discoverable by registering this marker service using the Whiteboard pattern.

Integration of third-party REST interfaces with the framework REST service on the implementation level might not always be possible since it requires knowledge about the underlying implementation and an extension mechanism on that level. Specific technologies such as servlets might support this but the REST service could as well be implemented without the use of a supporting abstraction layer and not offer extensibility.

Using this marker service, the REST service includes the advertised service in the Extensions Resource, allowing clients to discover it and use the extension's functionality.

137.10.2.1 **public static final String NAME = "org.osgi.rest.name"**

This service property describes the package name of the technology manageable by this REST API extension. Services specified in OSGi specifications must use their canonical package name as the name. Third-party technologies should also use their package names. The type of this property is `java.lang.String` and the property is mandatory.

137.10.2.2 public static final String SERVICE = "org.osgi.rest.service"

This service property refers to the id of the service the REST API extension provides management capabilities for. This can be useful if more than one service of a given type is present in the framework. For example if more than one Http Service is available this property is used to associate a REST extension managing the Http Service with a specific service instance. The type of the property is `java.lang.Long` and the property is optional; if the REST extension is not directly associated with a service in the service registry, the property should not be set.

137.10.2.3 public static final String URI_PATH = "org.osgi.rest.uri.path"

This service property describes a URI to the REST extension on this local machine. It is either an fully qualified URI with a different port if no integration with the framework REST service is possible or a relative URI implicitly using the same port if integration is possible. The type of this property is `java.lang.String` and the property is mandatory.

137.11 org.osgi.service.rest.client

Rest Service Client Package Version 1.0.

137.11.1 Summary

- `RestClient` - A Java client API for a REST service endpoint.
- `RestClientFactory` - Factory to construct new REST client instances.

137.11.2 public abstract RestClient

A Java client API for a REST service endpoint.

Provides a Java client API for accessing and managing a remote OSGi framework through the REST API. Implementations of this interface will usually take the URL to the remote REST Management Service instance as an argument in their constructor. Further arguments might be needed, for example, if the cloud provider requires URL signing.

Provider Type Consumers of this API must not implement this type

137.11.2.1 public abstract BundleDTO getBundle(long id) throws Exception

id Addresses the bundle by its identifier.

- Retrieve the bundle representation for a given bundle Id.

Returns A `{@link BundleDTO}` for the requested bundle.

Throws `Exception`— An exception representing a failure in the underlying REST call.

137.11.2.2 public abstract BundleDTO getBundle(String bundlePath) throws Exception

bundlePath Addresses the bundle by its URI path.

- Retrieve the bundle representation for a given bundle path.

Returns A `{@link BundleDTO}` for the requested bundle.

Throws `Exception`— An exception representing a failure in the underlying REST call.

137.11.2.3 public abstract Map<String, String> getBundleHeaders(long id) throws Exception

id Addresses the bundle by its identifier.

- Get the header for a bundle given by its bundle Id.

Returns Returns the map of headers entries.

Throws Exception— An exception representing a failure in the underlying REST call.

137.11.2.4 public abstract Map<String, String> getBundleHeaders(String bundlePath) throws Exception

bundlePath Addresses the bundle by its URI path.

- Get the header for a bundle given by its URI path.

Returns Returns the map of headers entries.

Throws Exception— An exception representing a failure in the underlying REST call.

137.11.2.5 public abstract Collection<String> getBundlePaths() throws Exception

- Get the bundles currently installed on the managed framework.

Returns Returns a collection of the bundle URIs in the form of Strings. The URIs are relative to the REST API root URL and can be used to retrieve bundle representations.

Throws Exception— An exception representing a failure in the underlying REST call.

137.11.2.6 public abstract Collection<BundleDTO> getBundles() throws Exception

- Get the bundle representations for all bundles currently installed in the managed framework.

Returns Returns a collection of BundleDTO objects.

Throws Exception— An exception representing a failure in the underlying REST call.

137.11.2.7 public abstract BundleStartLevelDTO getBundleStartLevel(long id) throws Exception

id Addresses the bundle by its identifier.

- Get the start level for a bundle given by its bundle Id.

Returns Returns a {@link BundleStartLevelDTO} describing the current start level of the bundle.

Throws Exception— An exception representing a failure in the underlying REST call.

137.11.2.8 public abstract BundleStartLevelDTO getBundleStartLevel(String bundlePath) throws Exception

bundlePath Addresses the bundle by its URI path.

- Get the start level for a bundle given by its URI path.

Returns Returns a {@link BundleStartLevelDTO} describing the current start level of the bundle.

Throws Exception— An exception representing a failure in the underlying REST call.

137.11.2.9 public abstract int getBundleState(long id) throws Exception

id Addresses the bundle by its identifier.

- Get the state for a given bundle Id.

Returns Returns the current bundle state as defined in (@link org.osgi.framework.Bundle).

Throws Exception— An exception representing a failure in the underlying REST call.

137.11.2.10 public abstract int getBundleState(String bundlePath) throws Exception

bundlePath Addresses the bundle by its URI path.

- Get the state for a given bundle path.

Returns Returns the current bundle state as defined in (@link org.osgi.framework.Bundle).

Throws Exception— An exception representing a failure in the underlying REST call.

- 137.11.2.11** **public abstract FrameworkStartLevelDTO getFrameworkStartLevel() throws Exception**
- Retrieves the current framework start level.
- Returns* Returns the current framework start level in the form of a {@link FrameworkStartLevelDTO}.
- Throws* Exception— An exception representing a failure in the underlying REST call.
- 137.11.2.12** **public abstract Collection<String> getServicePaths() throws Exception**
- Gets a collection of URI paths to all installed services.
- Returns* Returns a collection of URI paths to the installed services.
- Throws* Exception— An exception representing a failure in the underlying REST call.
- 137.11.2.13** **public abstract Collection<String> getServicePaths(String filter) throws Exception**
- filter* Passes a filter to restrict the result set.
- Gets a collection of URI paths to all installed services.
- Returns* Returns a collection of URI paths to the installed services.
- Throws* Exception— An exception representing a failure in the underlying REST call.
- 137.11.2.14** **public abstract ServiceReferenceDTO getServiceReference(long id) throws Exception**
- id* Addresses the service by its identifier.
- Get the service representation for a service given by its service Id.
- Returns* The service representation as {@link ServiceReferenceDTO}.
- Throws* Exception— An exception representing a failure in the underlying REST call.
- 137.11.2.15** **public abstract ServiceReferenceDTO getServiceReference(String servicePath) throws Exception**
- servicePath* Addresses the service by its URI path.
- Get the service representation for a service given by its URI path.
- Returns* The service representation as {@link ServiceReferenceDTO}.
- Throws* Exception— An exception representing a failure in the underlying REST call.
- 137.11.2.16** **public abstract Collection<ServiceReferenceDTO> getServiceReferences() throws Exception**
- Get the service representations for all services.
- Returns* Returns the service representations in the form of {@link ServiceReferenceDTO} objects.
- Throws* Exception— An exception representing a failure in the underlying REST call.
- 137.11.2.17** **public abstract Collection<ServiceReferenceDTO> getServiceReferences(String filter) throws Exception**
- filter* Passes a filter to restrict the result set.
- Get the service representations for all services.
- Returns* Returns the service representations in the form of {@link ServiceReferenceDTO} objects.
- Throws* Exception— An exception representing a failure in the underlying REST call.
- 137.11.2.18** **public abstract BundleDTO installBundle(String location) throws Exception**
- location* Passes the location string to retrieve the bundle content from.
- Install a new bundle given by an externally reachable location string, typically describing a URL.
- Returns* Returns the {@link BundleDTO} of the newly installed bundle.
- Throws* Exception— An exception representing a failure in the underlying REST call.

137.11.2.19 public abstract BundleDTO installBundle(String location, InputStream in) throws Exception

location Passes the location string to be used to install the new bundle.

in Passes the input stream to a bundle.

- Install a new bundle given by an {@link InputStream} to a bundle content.

Returns Returns the {@link BundleDTO} of the newly installed bundle.

Throws Exception— An exception representing a failure in the underlying REST call.

137.11.2.20 public abstract void setBundleStartLevel(long id, int startLevel) throws Exception

id Addresses the bundle by its identifier.

startLevel The target start level.

- Set the start level for a bundle given by its bundle Id.

Throws Exception— An exception representing a failure in the underlying REST call.

137.11.2.21 public abstract void setBundleStartLevel(String bundlePath, int startLevel) throws Exception

bundlePath Addresses the bundle by its URI path.

startLevel The target start level.

Throws Exception— An exception representing a failure in the underlying REST call.

137.11.2.22 public abstract void setFrameworkStartLevel(FrameworkStartLevelDTO startLevel) throws Exception

startLevel set the framework start level to this target.

- Sets the current framework start level.

Throws Exception— An exception representing a failure in the underlying REST call.

137.11.2.23 public abstract void startBundle(long id) throws Exception

id Addresses the bundle by its identifier.

- Start a bundle given by its bundle Id.

Throws Exception— An exception representing a failure in the underlying REST call.

137.11.2.24 public abstract void startBundle(String bundlePath) throws Exception

bundlePath Addresses the bundle by its URI path.

- Start a bundle given by its URI path.

Throws Exception— An exception representing a failure in the underlying REST call.

137.11.2.25 public abstract void startBundle(long id, int options) throws Exception

id Addresses the bundle by its identifier.

options Passes additional options as defined in {@link org.osgi.framework.Bundle#start(int)}

- Start a bundle given by its bundle Id.

Throws Exception— An exception representing a failure in the underlying REST call.

137.11.2.26 public abstract void startBundle(String bundlePath, int options) throws Exception

bundlePath Addresses the bundle by its URI path.

options Passes additional options as defined in {@link org.osgi.framework.Bundle#start(int)}

- Start a bundle given by its URI path.

Throws Exception— An exception representing a failure in the underlying REST call.

137.11.2.27 public abstract void stopBundle(long id) throws Exception

id Addresses the bundle by its identifier.

- Stop a bundle given by its bundle Id.

Throws Exception— An exception representing a failure in the underlying REST call.

137.11.2.28 public abstract void stopBundle(String bundlePath) throws Exception

bundlePath Addresses the bundle by its URI path.

- Stop a bundle given by its URI path.

Throws Exception— An exception representing a failure in the underlying REST call.

137.11.2.29 public abstract void stopBundle(long id, int options) throws Exception

id Addresses the bundle by its identifier.

options Passes additional options as defined in {@link org.osgi.framework.Bundle#stop(int)}

- Stop a bundle given by its bundle Id.

Throws Exception— An exception representing a failure in the underlying REST call.

137.11.2.30 public abstract void stopBundle(String bundlePath, int options) throws Exception

bundlePath Addresses the bundle by its URI path.

options Passes additional options as defined in {@link org.osgi.framework.Bundle#stop(int)}

- Stop a bundle given by its URI path.

Throws Exception— An exception representing a failure in the underlying REST call.

137.11.2.31 public abstract BundleDTO uninstallBundle(long id) throws Exception

id Addresses the bundle by its identifier.

- Uninstall a bundle given by its bundle Id.

Returns Returns the {@link BundleDTO} of the uninstalled bundle.

Throws Exception— An exception representing a failure in the underlying REST call.

137.11.2.32 public abstract BundleDTO uninstallBundle(String bundlePath) throws Exception

bundlePath Addresses the bundle by its URI path.

- Uninstall a bundle given by its URI path.

Returns Returns the {@link BundleDTO} of the uninstalled bundle.

Throws Exception— An exception representing a failure in the underlying REST call.

137.11.2.33 public abstract BundleDTO updateBundle(long id) throws Exception

id Addresses the bundle by its identifier.

- Updates a bundle given by its bundle Id using the bundle-internal update location.

Returns Returns the {@link BundleDTO} of the updated bundle.

Throws Exception— An exception representing a failure in the underlying REST call.

137.11.2.34 public abstract BundleDTO updateBundle(long id, String url) throws Exception

id Addresses the bundle by its identifier.

url The URL whose content is to be used to update the bundle.

- Updates a bundle given by its URI path using the content at the specified URL.

Returns Returns the {@link BundleDTO} of the updated bundle.

Throws Exception— An exception representing a failure in the underlying REST call.

137.11.2.35 **public abstract BundleDTO updateBundle(long id, InputStream in) throws Exception**

id Addresses the bundle by its identifier.

in Passes an input stream to the new bundle content.

- Updates a bundle given by its bundle Id and passing the new bundle content in the form of an {@link InputStream}.

Returns Returns the {@link BundleDTO} of the updated bundle.

Throws Exception— An exception representing a failure in the underlying REST call.

137.11.3 **public abstract RestClientFactory**

Factory to construct new REST client instances. Each instance is specific to a REST service endpoint.

Implementations can choose to extend this interface to add additional creation methods, where additional arguments are needed for request signing, etc.

In OSGi environments, this factory is registered as a service.

Provider Type Consumers of this API must not implement this type

137.11.3.1 **public abstract RestClient createRestClient(URI uri)**

uri The URI to the REST service endpoint.

- Create a new REST client instance.

Returns A new REST client instance for the specified REST service endpoint.

137.12 **JavaScript Client API**

REST JavaScript Client API Version 1.0

137.12.1 **Summary**

- `OSGiRestClient` - A JavaScript client API for accessing and managing a remote OSGi framework through the REST API.
- `OSGiRestCallback` - Callback object provided to the `OSGiRestClient` functions. Invoked on completion of the remote invocation.

JavaScript does not support the concept of interfaces and therefore implementations of the JavaScript client specification can provide objects of any type as long as they conform to the to the signatures described in this specification.

To facilitate documenting the JavaScript APIs *Web IDL* is used; see [2] *Web IDL* . This clarifies the accepted arguments and return types for otherwise untyped functions. Web IDL is only used for documentation purposes and has no bearing on the implementation of this API.

Note: some data types in Web IDL have slightly different names than commonly used in languages like Java or JavaScript. For example a String is called DOMString and the equivalent of a Java long is called long long. Additionally, when a representation as defined in this specification is passed to one of the JavaScript client APIs this representation is provided as a JavaScript object. Following the recommendations for mapping these to Web IDL, these JavaScript Object parameters are described using the dictionary data type. For more information see the Web IDL specification.

137.12.2 interface OSGiRestClient

Provides a JavaScript client API for accessing and managing a remote OSGi framework through the REST API. Implementations will provide a proprietary constructor to create objects of this signature. Once created the object can be used from JavaScript environments to manage the framework.

137.12.2.1 void getBundle((DOMString or long long) bundle, OSGiRestCallback cb)

bundle The bundle, either the numeric bundle ID or the bundle URI path.

- cb* The callbacks invoked on completion of the remote invocation. On success the success() callback is invoked with the Bundle representation as JavaScript object.
- Get the Bundle representation of a specific bundle.

137.12.2.2 void getBundleHeader((DOMString or long long) bundle, OSGiRestCallback cb)

bundle The bundle, either the numeric bundle ID or the bundle URI path.

- cb* The callbacks invoked on completion of the remote invocation. On success the success() callback is invoked with the Bundle Header representation as JavaScript object.
- Get the Bundle Header representation of a specific bundle.

137.12.2.3 void getBundleRepresentations(OSGiRestCallback cb)

- cb* The callbacks invoked on completion of the remote invocation. On success the success() callback is invoked with the Bundle Representations List representation as JavaScript object.
- List the bundles details.

137.12.2.4 void getBundles(OSGiRestCallback cb)

- cb* The callbacks invoked on completion of the remote invocation. On success the success() callback is invoked with the Bundle List representation as JavaScript object.
- List the bundles.

137.12.2.5 void getBundleStartLevel((DOMString or long long) bundle, OSGiRestCallback cb)

bundle The bundle, either the numeric bundle ID or the bundle URI path.

- cb* The callbacks invoked on completion of the remote invocation. On success the success() callback is invoked with the Bundle Start Level representation as JavaScript object.
- Get the Bundle Start Level representation of a specific bundle.

137.12.2.6 void getBundleState((DOMString or long long) bundle, OSGiRestCallback cb)

bundle The bundle, either the numeric bundle ID or the bundle URI path.

- cb* The callbacks invoked on completion of the remote invocation. On success the success() callback is invoked with the Bundle State representation as JavaScript object.
- Get the Bundle State representation of a specific bundle.

137.12.2.7 void getFrameworkStartLevel(OSGiRestCallback cb)

- cb* The callbacks invoked on completion of the remote invocation. On success the success() callback is invoked with the Framework Start Level representation as JavaScript object.
- Obtain the Framework Start Level.

137.12.2.8 void getService((DOMString or long long) service, OSGiRestCallback cb)

service The service, either the numeric service ID or the service URI path.

- cb* The callbacks invoked on completion of the remote invocation. On success the success() callback is invoked with the Service representation as JavaScript object.

- Get a service representation.

137.12.2.9 void getServiceRepresentations(OSGiRestCallback cb)

cb The callbacks invoked on completion of the remote invocation. On success the success() callback is invoked with the Service Representations List representation as JavaScript object.

- Get all services representations.

137.12.2.10 void getServices(OSGiRestCallback cb)

cb The callbacks invoked on completion of the remote invocation. On success the success() callback is invoked with the Service List representation as JavaScript object.

- Get all services URIs.

137.12.2.11 void installBundle((DOMString or ArrayBuffer) bundle, OSGiRestCallback cb)

bundle The Bundle to install, either represented as a URL or as an ArrayBuffer of

cb The callbacks invoked on completion of the remote invocation. On success the success() callback is invoked with the Bundle representation of the newly installed Bundle. This parameter is optional.

- Install a bundle from a URI or by value.

137.12.2.12 void setBundleStartLevel((DOMString or long long) bundle, dictionary bsl, OSGiRestCallback cb)

bundle The bundle, either the numeric bundle ID or the bundle URI path.

bsl A Bundle Start Level representation dictionary with the desired state.

cb The callbacks invoked on completion of the remote invocation. On success the success() callback is invoked with the resulting Framework Start Level representation as JavaScript object. This parameter is optional.

- Change the Framework Start Level and/or initial bundle start level.

137.12.2.13 void setBundleState((DOMString or long long) bundle, dictionary state, OSGiRestCallback cb)

bundle The bundle, either the numeric bundle ID or the bundle URI path.

state Bundle State representation dictionary with the desired state.

cb The callbacks invoked on completion of the remote invocation. On success the success() callback is invoked with the resulting Bundle Start Level representation as JavaScript object. This parameter is optional.

- Change the Bundle Start Level and/or other options defined in the Bundle Start Level representation.

137.12.2.14 void setFrameworkStartLevel(dictionary fwsl, OSGiRestCallback cb)

fwsl Framework Start Level representation dictionary with the desired state.

cb The callbacks invoked on completion of the remote invocation. On success the success() callback is invoked with the resulting Framework Start Level representation as JavaScript object. This parameter is optional.

- Change the Framework Start Level and/or initial bundle start level.

137.12.2.15 void startBundle((DOMString or long long) bundle, long options, OSGiRestCallback cb)

bundle The bundle, either the numeric bundle ID or the bundle URI path.

options The options passed to the bundle's start method as a number. This parameter is optional.

cb The callbacks invoked on completion of the remote invocation. On success the success() callback is invoked with the Bundle State representation as JavaScript object. This parameter is optional.

- Start a bundle.

137.12.2.16 **void stopBundle((DOMString or long long) bundle, long options, OSGiRestCallback cb)**

bundle The bundle, either the numeric bundle ID or the bundle URI path.

options The options passed to the bundle's start method as a number. This parameter is optional.

cb The callbacks invoked on completion of the remote invocation. On success the success() callback is invoked with the Bundle State representation as JavaScript object. This parameter is optional.

- Stop a bundle.

137.12.2.17 **void uninstallBundle((DOMString or long long) bundle, OSGiRestCallback cb)**

bundle The bundle, either the numeric bundle ID or the bundle URI path.

cb The callbacks invoked on completion of the remote invocation. On success the success() callback is invoked with the Bundle representation of the uninstalled Bundle. This parameter is optional.

- Uninstall a bundle.

137.12.2.18 **void updateBundle((DOMString or long long) bundle, (DOMString or ArrayBuffer) updated, OSGiRestCallback cb)**

bundle The bundle, either the numeric bundle ID or the bundle URI path.

updated The Bundle to update, either represented as a URL or as an ArrayBuffer of

cb The callbacks invoked on completion of the remote invocation. On success the success() callback is invoked with the Bundle representation of the updated Bundle. This parameter is optional.

- Update a bundle from a URI or by value.

137.12.3 **callback interface OSGiRestCallback**

Objects implementing this signature are provided by users of the OSGiRestClient as callbacks. One of the callback functions is invoked on completion of a REST invocation.

137.12.3.1 **void success(object response)**

response The result of the invocation. The type of this parameter is depends on the function being invoked. It can be found in the documentation of the function.

- Called when the invocation completes successfully.

137.12.3.2 **void failure(short httpCode, object response)**

httpCode The HTTP code returned. If no HTTP code is associated with the failure this parameter is set to -1.

response The failure response.

- Called when the invocation failed.

137.13 **References**

- [1] *Framework Filter Syntax*
OSGi Core, Chapter 3.2.7 Filter Syntax
- [2] *Web IDL*
<https://www.w3.org/TR/WebIDL/>
- [3] *OSGi XML Schemas*
<https://docs.osgi.org/xmlns/>

- [4] *Whiteboard Pattern*
<https://docs.osgi.org/whitepaper/whiteboard-pattern/>

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT

End Of Document

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT